Efficient Sumcheck

A sumcheck library built for Arkworks grounded in textbook formalization.

The Problem

The sumcheck protocol is **one protocol** parameterized by the polynomial shape. The old library treated each shape as a separate implementation:

- 3 protocol runners, 2 verifiers, 2 return types
- 10+ public entry points with inconsistent signatures
- Adding a new polynomial shape meant duplicating the entire stack

This made integration hard (WHIR needed hooks, GKR needed partial execution, Jolt needed a different variable ordering) and every new feature touched every runner.

10,800 LOC. The code was correct. The architecture was not.

The Goal

Reduce complexity without losing functionality.

Metric	Before	After
Lines of code	10,800	7,448
Source files	~100	43
Public entry points	10+	4
Order strategies	4	2 (MSB, LSB)
Protocol runners	3	1
Verifiers	2	1
Return types	2	1
Tests	69	63
Clippy warnings	46	0

The Authoritative Source

Justin Thaler, *Proofs, Arguments, and Zero-Knowledge*, Chapter 4.

Proposition 4.1. Given a v-variate polynomial g over F with degree at most d in each variable, the sum-check protocol proves `H = sum_{b in {0,1}^v} g(b)` in v rounds.

- Completeness error: 0
- Soundness error: <= v * d / |F|

Key insight: the protocol is *one* protocol parameterized by g. Three "different" sumchecks are three instantiations.

One Protocol, Three Instantiations

Use case	g	Degree	Reference
Multilinear	f_tilde (MLE)	1	Thaler S4.1
Inner product	f_tilde * g_tilde	2	Thaler S4.4
Coefficient	user-defined	d	Thaler S4.6

The protocol (transcript, consistency checks, challenges) is identical.

Only the prover's round polynomial computation changes.

This motivates **one runner + one trait**, not three functions.

The Prover Trait

```rust
pub trait SumcheckProver<F: SumcheckField> {
    fn degree(&self) -> usize;
    fn round(&mut self, challenge: Option<F>) -> Vec<F>;
    fn finalize(&mut self, last_challenge: F);
    fn final_value(&self) -> F;
}
```

Lifecycle:

```
round(None)          -> g_0 evaluations     // round 0
round(Some(r_0))     -> g_1 evaluations     // fold with r_0, compute g_1
...
round(Some(r_{v-2})) -> g_{v-1} evaluations // fold, compute
finalize(r_{v-1})                            // apply last challenge
final_value()        -> g(r_0, ..., r_{v-1}) // oracle value
```

The Protocol Runner

```
pub fn sumcheck<F: SumcheckField, T: ProverTranscript<F>>(
    prover: &mut impl SumcheckProver<F>,
    num_rounds: usize,
    transcript: &mut T,
    hook: impl FnMut(usize, &mut T),
) -> SumcheckProof<F>
```

One function handles:

- Full sumcheck (num_rounds = v)
- Partial sumcheck (num_rounds < v) for GKR and WHIR
- Per-round hooks for proof-of-work grinding
- Any polynomial degree (degree is prover-reported)

The runner never inspects prover internals.

The Verifier

```rust
pub fn sumcheck_verify<F: SumcheckField, T: VerifierTranscript<F>>(
    claimed_sum: F,
    expected_degree: usize,
    num_rounds: usize,
    transcript: &mut T,
    hook: impl FnMut(usize, &mut T) -> Result<(), SumcheckError>,
) -> Result<SumcheckResult<F>, SumcheckError>
```

- Receives `d` values per round (EvalsInfty wire format — next slide)
- Consistency `g_j(0) + g_j(1) = claim` is structural, not a runtime check
- Evaluates g_j(r_j) via polynomial reconstruction (any degree)
- Returns `SumcheckResult { challenges, final_claim }`

The verifier doesn't know g (Thaler Remark 4.2), so the oracle check is the caller's responsibility. The API returns `final_claim` directly — the caller handles it according to their protocol:

```rust
assert_eq!(result.final_claim, proof.final_value) // standalone
next_claim = result.final_claim;                   // WHIR, GKR (next layer)
```

Wire Format — EvalsInfty

All provers emit round polynomials in the **EvalsInfty** format: `d` values per round for a degree-`d` polynomial, one fewer than a full evaluation table.

d	Wire	Recovered by verifier
1	`[h(0)]`	`h(1) = claim − h(0)`
≥ 2	`[h(0), h(∞), h(2), …, h(d−1)]`	`h(1) = claim − h(0)`

`h(∞)` is the leading coefficient (coefficient of `x^d`).

- **Cheapest for product-structured summands** — the leading coefficient is a difference-of-shifted-evaluations form (BDDT25, Algorithm 3).
- **Consistency is structural.** `h(0) + h(1) = claim` is enforced by the wire format rather than a runtime check — a dishonest prover's misbehavior surfaces at the caller's oracle check rather than mid-protocol.

Unified Proof Type

```rust
pub struct SumcheckProof<F: SumcheckField> {
    pub round_polys: Vec<Vec<F>>,  // g_j in EvalsInfty wire format
    pub challenges: Vec<F>,        // r_1, ..., r_v
    pub final_value: F,            // g(r_1, ..., r_v)
}
```

Replaces both `Sumcheck<F>` and `ProductSumcheck<F>`.

Prover-specific post-state (e.g., `(f(r), g(r))` for inner product) lives on the prover via `&mut P` ownership, not in the proof.

Concrete Provers

Prover	Degree	Post-state
`MultilinearProver`	1	—
`InnerProductProver`	2	`final_evaluations() -> (F, F)`
`CoefficientProver`	d	—
`GkrProver`	2	`claimed_w_values() -> (F, F)`
`EqFactoredProver` *(available)*	2	`final_factors() -> (F, F)`

MSB and LSB variants exist for the first three; `GkrProver` and `EqFactoredProver` are MSB-only.

Same runner. Same verifier. Same proof type.

The Fold Primitive (Lemma 4.3)

```
new[k] = v[k] + weight * (v[k + L/2] - v[k])
```

- Half-split (MSB) layout: fold the topmost variable each round
- Matches Thaler eq. 4.13 directly
- Non-power-of-two: implicit zero padding on the high half
- SIMD-accelerated for Goldilocks (transparent, zero overhead otherwise)
- Exposed publicly for WHIR's `multilinear_fold`

SIMD Acceleration

Goldilocks field (p = 2^64 - 2^32 + 1):

Backend	Width	Platform
NEON	2-wide	aarch64 (Apple M-series, Graviton)
AVX-512 IFMA	8-wide	x86_64 (Sapphire Rapids)

Two paths to SIMD:

Arkworks (automatic): blanket impl detects Goldilocks from modulus. LLVM const-folds the branch. Zero overhead on non-Goldilocks.

Non-arkworks (explicit): implement `SimdRepr` with `zerocopy` bounds:

```
pub trait SimdRepr:
    SumcheckField + zerocopy::IntoBytes + zerocopy::FromBytes
{
    fn modulus() -> u64;  // GOLDILOCKS_P for SIMD
}
```

Layout safety is **compiler-verified** via zerocopy derives. No `unsafe`.

Generic Field Trait

```rust
pub trait SumcheckField:
    Copy + Send + Sync + PartialEq + Debug
    + Add + Sub + Mul + Neg
    + AddAssign + SubAssign + MulAssign
    + Sum + 'static
{
    const ZERO: Self;
    const ONE: Self;
    fn from_u64(val: u64) -> Self;
    fn inverse(&self) -> Option<Self>;
    fn extension_degree() -> u64;
    fn _simd_field_config() -> Option<SimdFieldConfig>;
}
```

- **Not coupled to arkworks.** Any field-like type works.
- Blanket impl for `ark_ff::Field` behind `feature = "arkworks"` (default-on).
- Non-arkworks Goldilocks can opt into SIMD by overriding `_simd_field_config()`.

Cross-Field Support (BF -> EF) *(TODO)*

Evaluations in base field BF (e.g., Goldilocks).
Challenges from extension field EF (e.g., Goldilocks^3) for soundness.

```
pub trait ExtensionOf<BF: SumcheckField>: SumcheckField + From<BF> {}
```

The transition is prover-internal:

- Round 0: compute over BF
- `round(Some(r_1))`: lift BF -> EF, continue in EF
- Protocol runner and verifier never see BF

WHIR Integration (PR)

```
for round_config in &self.round_configs {
    round_config.committer.commit(&a);
    update_covector(&mut b, &stir_challenges);

    let proof = sumcheck(
        &mut InnerProductProver::new(a, b),
        round_config.folding_factor,     // partial!
        &mut transcript,
        |_, t| round_config.round_pow.prove(t),  // hook!
    );

    a = prover.a();  // post-state access
    b = prover.b();
}
```

Three features exercised: partial execution, per-round hook, post-state.

GKR Integration

`GkrProver` implements `SumcheckProver` for the GKR round polynomial:

```
f_r(b, c) = add_i(r, b, c) · (W(b) + W(c)) + mult_i(r, b, c) · (W(b) · W(c))
```

```
let mut prover = GkrProver::new(add_evals, mult_evals, w_evals);
let proof = sumcheck(&mut prover, 2 * k, &mut t, noop_hook);
let (w_b, w_c) = prover.claimed_w_values();  // for reduce-to-one
```

Same runner, same verifier, same proof type. GKR is just another prover.

Reduce-to-one is a separate composable sub-protocol (Thaler §4.5.2).

Eq-Factored Sumcheck *(available)*

Proves `H = Σ_{x ∈ {0,1}^v} eq(w, x) · p(x)` for fixed `w ∈ F^v` and multilinear `p`. Degree 2. Shows up in lookup arguments and any reduction that couples a public point to a witness polynomial via the multilinear equality predicate.

```
let mut prover = EqFactoredProver::new(w, p_evals);
let proof = sumcheck(&mut prover, v, &mut t, noop_hook);
let (p_r, eq_wr) = prover.final_factors();
// proof.final_value == p_r · eq_wr
```

Split-Value Optimization (BDDT25 Algorithm 5).

`eq` factors over any split of the variables:

```
eq(w, x) = eq(w_L, x_L) · eq(w_R, x_R)
```

`eq(w, ·)` is stored as two half-tables of `2^{v/2}` entries each rather than a single `2^v` table, and round-polynomial contributions are streamed through a nested-sum kernel without materializing the product.

BatchedEqFactoredProver *(future possibility)*

WHIR's covector is an RLC of equality polynomials:

```
b(x) = Σ_i α_i · eq(ρ_i, x)
```

A batched split-value prover would take `Vec<(ρ_i, α_i)>` + witness `a` instead of a materialized covector, store each `eq(ρ_i, ·)` in split form, and stream all `k` factors through the nested-sum kernel per round.

	Time	Memory
Current WHIR (`InnerProductProver` + materialized `b`)	`O(k · 2^v)`	`O(2^v)`
Batched split-value	`O(k · 2^v)`	`O(k · 2^{v/2})`

- **Memory win** when `k < 2^{v/2}` — typical for WHIR (`v ~ 20`, `k` in the dozens).
- **Time essentially unchanged.** You skip the upfront covector build but pay `k×` more per sumcheck round. Split-value is a

Two Orthogonal Axes

Prover design has two independent choices:

Space strategy -- how much memory to budget:

- Time: O(2^v) -- hold all evaluations
- Blendy: O(2^k) -- partition into stages, recompute per stage
- Space: O(v) -- recompute everything (academic only)

Variable ordering -- which variable to fold each round:

- MSB (half-split): pairs `(v[k], v[k+L/2])` -- in-memory and seekable streams
- LSB (pair-split): pairs `(v[2k], v[2k+1])` -- sequential/incremental streams

These are orthogonal. Blendy + MSB and Blendy + LSB are both valid.

Streaming Taxonomy

Scenario	Data	Access	Ordering	Example
In-memory	Full table in RAM	Random	MSB	WHIR
Random-access stream	On disk, too big for RAM	Seekable	MSB	Large witness (mmap)
Sequential stream	Generated incrementally	Forward-only	LSB	Jolt CPU trace

Random-access (mmap'd SSD): data exists but doesn't fit in RAM.
MSB reads two contiguous half-table regions -- good cache behavior.

Sequential (Jolt trace): evaluations arrive in index order.
LSB pairs `(f[2k], f[2k+1])` are immediately available --
folding begins before the full table exists.

Both streaming cases use blendy. The ordering choice depends on the data source.

Blendy Stage Scheduling (BCFFMMZ25) *(TODO)*

Jolt's `HalfSplitSchedule` uses **cost-model-driven, non-uniform windows**:

```
w(i) = round(ratio * i)    where ratio = ln(2) / ln((d+1)/2)
```

Windows grow with round number: early rounds (large hypercube) get small windows; later rounds (small residual) get large windows.

Degree	Ratio	Window sequence
2	1.71	1, 2, 5, 14, ...
3	1.00	1, 1, 2, 3, 4, ...
4	0.76	1, 1, 1, 2, 2, 3, ...

Two-phase structure:

1. Streaming phase (first half): cost-optimal windows, one trace pass per window
2. Linear phase (second half): materialized mode, every round is its own window

Parameterized by `StreamingSchedule` trait -- not a fixed constant.

Jolt Compatibility *(description of possible integration)*

Jolt's `SumcheckInstanceProver` trait:

```
fn compute_message(&mut self, round: usize, claim: F) -> UniPoly<F>;
fn ingest_challenge(&mut self, r_j: F::Challenge, round: usize);
fn finalize(&mut self);
```

Our `SumcheckProver` maps cleanly via adapter:

```
fn compute_message(&mut self, round: usize, _claim: F) -> UniPoly<F> {
    let challenge = self.pending.take().map(Into::into);
    UniPoly::from_evals(&self.inner.round(challenge))
}
fn ingest_challenge(&mut self, r_j: F::Challenge, _round: usize) {
    self.pending = Some(r_j);
}
```

LSB multilinear prover + this adapter = drop-in replacement for Jolt.

What We Deleted

Module	LOC	Why
multilinear/provers/	1,190	Old Prover trait scaffolding
multilinear_product/provers/	1,175	Same
order_strategy/	315	4 strategies -> MSB only
SIMD dispatch (product paths)	~500	Superseded by fused MSB kernels
messages/, interpolation/	327	Unused / graycode-coupled
Old test harness	~800	Legacy prover tests
prover/core.rs	24	Old trait definition

Total: 4,518 lines deleted across 86 files.

Advanced Optimizations Fit the Trait

From Bagad-Dao-Domb-Thaler, *Sumcheck Optimizations* (ePrint 2025/1117):

Optimization	Changes protocol?	Changes wire format?
LinearTime (Alg 1)	No	No
SqrtSpace (Alg 2)	No	No
SmallValue (Alg 3-4)	No	No
EqPoly (Alg 5-6)	No	No
Univariate skip	No	No

All optimizations live below the trait boundary.

The protocol runner, verifier, and transcript are untouched.
Each optimization is a different `SumcheckProver` implementation.

Feature Gate Architecture

```
[features]
default = ["arkworks", "parallel", "simd"]

arkworks = ["ark-ff", "ark-poly", "ark-serialize",
            "ark-std", "spongefish"]
parallel = ["rayon", "ark-ff?/parallel", ...]
simd     = []
```

- `--no-default-features`: pure `SumcheckField` library, no arkworks, no SIMD
- `--features arkworks`: blanket impl for `ark_ff::Field`
- `--features parallel`: rayon parallelism for fold and round computation
- `--features simd`: SIMD backends (NEON, AVX-512 IFMA) for Goldilocks
- SIMD dispatch is const-folded by LLVM -- zero overhead when field doesn't match

Benchmarking Strategy — TODO

Layer 1: Kernel Throughput

- fold elements/sec vs memory bandwidth ceiling
- round() overhead on top of fold
- BF->EF promotion cliff

Layer 2: Protocol Scaling

- Full sumcheck: time vs 2^v on log-log axes
- Matrix: {time, blendy} x {ml, lP} x {F64, F64Ext3} x {2^16..2^24}
- Key metric: **time per element per round**

Layer 3: Downstream Integration

- WHIR sumcheck bench as acceptance gate
- Regression detection, not absolute performance

CI Benchmark Infrastructure — TODO

```yaml
# .github/workflows/bench.yml
- name: Detect CPU features
  run: |
    if grep -q avx512ifma /proc/cpuinfo; then
      echo "rustflags=-C target-feature=+avx512ifma"
    fi

- name: Run benchmarks
  run: cargo bench --bench sumcheck -- --output-format bencher

- uses: benchmark-action/github-action-benchmark@v1
  with:
    alert-threshold: "115%"
    auto-push: true  # on main
```

- Auto-detects AVX-512 IFMA / NEON / scalar
- Results to gh-pages with trendlines
- 15% regression = alert + PR comment

Current Benchmark Matrix

18 benchmark points across 6 groups:

```
multilinear/F64:      2^16, 2^20, 2^24
multilinear/F64Ext3:  2^16, 2^20, 2^24
inner_product/F64:    2^16, 2^20, 2^24
inner_product/F64Ext3: 2^16, 2^20, 2^24
fold/F64:             2^16, 2^20, 2^24
fold/F64Ext3:         2^16, 2^20, 2^24
```

Criterion harness with `Throughput::Elements(n)` annotations.

~5 minutes on AVX-512 hardware.

Migration Table (internal)

Old effsc	New effsc
`multilinear_sumcheck(evals, t, hook)`	`sumcheck(&mut MultilinearProver::new(evals), v, t, hook)`
`inner_product_sumcheck(a, b, t, hook)`	`sumcheck(&mut InnerProductProver::new(a, b), v, t, hook)`
`multilinear_sumcheck_partial(...)`	`sumcheck(&mut prover, k, t, hook)`
`fold(values, weight)`	`fold(values, weight)` (unchanged)
`Sumcheck<F>`	`SumcheckProof<F>`
`ProductSumcheck<F>`	`SumcheckProof<F>` (unified)

Migration from arkworks-rs/sumcheck

arkworks-rs/sumcheck	effsc
`ListOfProductsOfPolynomials<F>`	Custom `RoundPolyEvaluator` + `CoefficientProver`
`GKRRoundSumcheck::prove(...)`	`sumcheck(&mut GkrProver::new(add, mult, w), ...)`
`GKRRoundSumcheck::verify(...)`	`sumcheck_verify(sum, 2, rounds, t, hook)`

See `docs/migration.md` for worked examples.

What's NOT in Scope

Explicit non-goals (from Thaler's framing):

- **Generic IP trait** -- sumcheck is specific, not an instance of a framework
- **Zero-knowledge** -- future `ZkSumcheckProver` wrapper (masking polynomials)
- **Batching** -- compose via random linear combinations externally
- **Reduce-to-one** -- separate sub-protocol (S4.5.2), not part of sumcheck

These can be added later without changing the core trait or runner.

Threat Model *(TODO)*

Sumcheck is public-coin: prover messages depend only on the polynomial and verifier challenges. This library implements plain (non-ZK) sumcheck.

- **Standalone** (`g` public): transcript reveals everything the prover computes on — timing side channels leak nothing beyond the transcript.
- **ZK embedding** (`g` encodes witness data; ZK supplied by surrounding commitments/masking): arithmetic runs on secrets. Transcript-observing adversaries are safe, but **resistance to local side-channel adversaries has not been formally verified**.

Fixed-size Montgomery multiplication is inherently data-independent but unaudited. No constant-time claim for arbitrary `SumcheckField` impls. Callers in that threat model must supply constant-time field operations.

See SECURITY.md for the full threat model and reporting policy.

Design Principles

1. **One protocol, one trait, many implementations.**
   Three polynomial shapes are three `SumcheckProver` impls, not three runners.

2. **The trait boundary is the optimization boundary.**
   Everything above it (runner, verifier, transcript) is fixed.
   Everything below it (fold, SIMD, table layout, multiplication tricks) is freedom.

3. **Partial execution is first-class.**
   `num_rounds < v` enables GKR and WHIR without special-casing.

4. **Post-state via ownership, not return types.**
   `&mut P` survives sumcheck; prover-specific accessors are type-safe.

5. **The verifier returns the final claim, not a verdict.**
   `sumcheck_verify` returns `SumcheckResult { challenges, final_claim }`.
   The oracle check is the caller's concern — standalone callers compare,
   composed callers (WHIR, GKR) pass `final_claim` to the next layer.

6. **Features are orthogonal layers.**

Next Steps

Item	Status	Notes
CoefficientProver	Done	MSB + LSB variants
GkrProver	Done	Reference impl, O(2^{2k})
LSB ordering	Done	All provers have MSB + LSB variants
WHIR integration	Done	PR #250
SECURITY.md	Done	Threat model, unsafe scope, disclosure policy
ark-sumcheck migration guide	Done	docs/migration.md
GkrProver O(2^k · k) optimization	Future	Incremental eq-polynomial bookkeeping
Blendy space strategy	Future	O(2^k) memory with stage scheduling
Jolt adapter	Future	Drop-in `SumcheckInstanceProver` impl
StreamingSchedule trait	Investigating	Cost-model windows (BCFFMMZ25)
Additional SIMD support	Future	M31, BabyBear, KoalaBear

Summary

The sum-check protocol is one protocol.

We made the code match that fact.

- 1 runner, 1 verifier, 1 proof type, 1 fold
- 7 concrete provers (multilinear, inner-product, coefficient, GKR × MSB/LSB)
- Generic over any field (not arkworks-specific)
- SIMD transparent for Goldilocks (AVX-512 IFMA, NEON)
- Integrated into WHIR and WARP with measured performance improvements
- Superset of arkworks-rs/sumcheck functionality (see docs/migration.md)
- Correctness-fuzzed against a formally verified oracle *(in progress)*
- All known optimizations fit below the trait boundary